SKILL-02

1. To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

1. Create Main Loop

- The main loop is the core part of a shell program.
- It continuously waits for commands from the user.
- The loop repeats until the user chooses to exit.
- The main loop generally contains prompt display, input reading, command processing, and execution.

2. Display Prompt

- The shell displays a prompt to indicate that it is ready to accept a command.
- A simple prompt can be:

ForgeOS$

- The prompt is displayed before every command entered by the user.

3. Read User Input

- The shell reads the command entered by the user.
- A buffer can be used to temporarily store the input.
- Functions such as fgets() can be used to read the complete command.
- The input is then processed by the shell.

4. Handle Exit Conditions

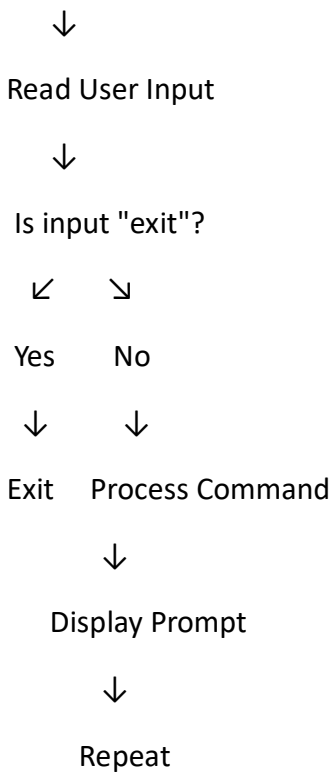
- The shell should provide a way for the user to terminate the main loop.
- A command such as exit can be used for this purpose.
- When the user enters exit, the loop terminates.
- Otherwise, the shell continues displaying the prompt and accepting commands.

5. Design Control Flow Diagram

Start

↓

Display Prompt



6. Test Interactive Loop

- Compile the shell program.
- Run the program in the terminal.
- Check whether the prompt is displayed correctly.
- Enter different commands and verify that the shell continues running.
- Enter exit and verify that the shell terminates.
- Test empty input and invalid commands to check the behavior of the loop.

To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction

1. Capture Keyboard Input

- The shell captures characters entered by the user through the keyboard.
- Each character is received and processed by the shell.
- The input is temporarily stored in an input buffer.

2. Handle Backspace

- The shell detects when the user presses the **Backspace** key.
- If characters are present in the buffer, the last character is removed.
- The displayed prompt is also updated to reflect the deletion.
- Backspace should not remove characters when the input buffer is empty.

3. Process Enter Key

- The **Enter** key indicates that the user has finished entering a command.
- When Enter is detected, the shell stops collecting characters.
- The contents of the input buffer are treated as the entered command.
- The buffer is then prepared for the next command.

4. Manage Input Buffer

- A character array is used to store the command entered by the user.
- Each new character is added to the buffer.
- A position or index is maintained to track the current length of the input.
- The buffer is cleared or reset after the command is processed.

5. Support Multi-Character Commands

- The shell should accept commands containing multiple characters.
- Examples include:
- ls
- pwd
- clear

exit

- Characters are stored one by one until the Enter key is pressed.
- The complete command is then processed by the shell.

6. Test User Interaction

- Run the shell program in the terminal.
- Enter a single-character or multi-character command.
- Press Backspace and verify that the last character is removed.

- Press Enter and verify that the complete command is accepted.
- Test multiple commands one after another.
- Test the exit command to verify that the shell terminates correctly.

Control Flow

Start

↓

Display Prompt

↓

Read Keyboard Character

↓

Is Backspace?

→ Yes → Remove Last Character

↓ No

Is Enter?

→ Yes → Process Command

↓ No

Store Character in Buffer

↓

Read Next Character